Report: Single-Model PyMC Collapsed Ridge RT Baseline (Seen Groups)

Bioinformatics Team

December 17, 2025

1 Executive summary

This document is the primary RT modeling direction going forward: a fast, single-model PyMC ridge-style baseline trained from sufficient statistics, with the intent to incorporate the best aspects of the older hierarchical RT model (pooling across species clusters, compounds, and chemistry) while keeping training scalable.

We pivoted away from the earlier "Stage-2" RT experiments and standardized on a simpler goal: **a single probabilistic model in PyMC** that is:

- competitive with per-group ridge RMSE on *seen* groups $g = (\texttt{species_cluster}, \texttt{comp_id})$, and
- provides meaningful credible intervals (full-covariance coefficient uncertainty).

The current baseline is a **collapsed Bayesian ridge** model with group-specific coefficients $\boldsymbol{\beta}_g$ (intercept + slopes) and global hyperparameters (σ_y, λ). The per-group coefficients are integrated out of the likelihood, enabling scalable inference from per-group sufficient statistics computed directly from the production CSV.

Key outcome (cap100 → realtest). Training on full cap100 and evaluating on realtest (seen groups only), the single-model PyMC baseline matches sklearn ridge RMSE while providing usable predictive intervals ($\text{cov}_{0.95} \approx 0.946$ for lib208 and ≈ 0.963 for lib209).

Why RMSE matches ridge (when λ is fixed). With the prior $\boldsymbol{w}_g \sim \mathcal{N}(\mathbf{0}, \sigma_y^2 \lambda^{-1} \boldsymbol{I})$ and a flat intercept prior, the posterior mean of $\boldsymbol{w}_g$ conditional on λ is exactly the ridge solution: $\hat{\boldsymbol{w}}_g = (\boldsymbol{X}_c^\top \boldsymbol{X}_c + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}_c^\top \boldsymbol{y}_c$. Therefore, when running `--lambda-mode fixed`, the PyMC model is intentionally ridge-equivalent for point predictions, so RMSE matches sklearn ridge up to numerical precision.

Where results live.

- Figures: `docs/models/images/rt_pymc_collapsed_ridge_report/`
- Metric JSON/CSV: `docs/models/results/rt_pymc_collapsed_ridge_report/`

2 Code inventory (what to use)

Baselines.

- Lasso (eslasso baseline): `scripts/pipelines/eval_rt_lasso_baseline_by_species_cluster.py`
- Ridge (sklearn per-group summaries): `scripts/pipelines/train_rt_stage1_coeff_summaries.py`

Single-model PyMC baseline.

- Train (collapsed ridge): `scripts/pipelines/train_rt_pymc_collapsed_ridge.py`

Shared evaluation and comparison.

- Evaluate any `Stage1CoeffSummaries`: `scripts/pipelines/eval_rt_coeff_summaries_by_spec`
- Compare-by-cluster plot: `scripts/pipelines/compare_rt_models_by_cluster.py`

3 Model specification

3.1 Data and grouping

We train and evaluate on production RT CSVs containing:

- response: $y = \texttt{rt}$,
- group key: $g = (\texttt{species_cluster}, \texttt{comp_id})$,
- run covariates: $\boldsymbol{x} \in \mathbb{R}^p$ from columns `IS*`, `RS*`, and optionally `ES_*`.

Optional global feature transform (pure reparameterization for stability):

$$\boldsymbol{x}' = (\boldsymbol{x} - \boldsymbol{\mu})\,\boldsymbol{R},$$

where $\boldsymbol{\mu}$ is the global mean over training rows (optional) and $\boldsymbol{R}$ is an optional orthonormal rotation (e.g. PCA rotation). The artifact stores $\boldsymbol{\mu}$ and $\boldsymbol{R}$ so evaluation applies the same transform.

3.2 Generative model (per group)

For each group g with n_g observations:

$$y_{gi} = b_g + \boldsymbol{x}_{gi}^\top \boldsymbol{w}_g + \varepsilon_{gi}, \quad \varepsilon_{gi} \sim \mathcal{N}(0, \sigma_y^2).$$

We use a ridge prior on slopes:

$$\boldsymbol{w}_g \mid \sigma_y^2, \lambda \sim \mathcal{N}\left(\boldsymbol{0}, \frac{\sigma_y^2}{\lambda} \boldsymbol{I}_p\right),$$

and treat the intercept b_g as unpenalized (equivalently: an improper flat prior on b_g).

Hyperpriors:

$$\sigma_y \sim \text{HalfNormal}(s_\sigma), \qquad \lambda \sim \begin{cases} \text{fixed} & (\texttt{--lambda-mode fixed}) \\ \text{LogNormal}(\mu_\lambda, s_\lambda) & (\texttt{--lambda-mode learn}) \end{cases}$$

3.3 Collapsed marginal likelihood (sufficient statistics)

Define per-group means and centered quantities:

$$\bar{y}_g = \frac{1}{n_g} \sum_i y_{gi}, \qquad \bar{\boldsymbol{x}}_g = \frac{1}{n_g} \sum_i \boldsymbol{x}_{gi},$$

$$\boldsymbol{y}_c = \boldsymbol{y}_g - \bar{y}_g \boldsymbol{1}, \qquad \boldsymbol{X}_c = \boldsymbol{X}_g - \boldsymbol{1} \bar{\boldsymbol{x}}_g^\top.$$

Sufficient statistics:

$$\boldsymbol{S}_{xx} = \boldsymbol{X}_c^\top \boldsymbol{X}_c \in \mathbb{R}^{p \times p}, \quad \boldsymbol{s}_{xy} = \boldsymbol{X}_c^\top \boldsymbol{y}_c \in \mathbb{R}^p, \quad s_{yy} = \boldsymbol{y}_c^\top \boldsymbol{y}_c \in \mathbb{R}.$$

Let $\boldsymbol{A} = \boldsymbol{S}_{xx} + \lambda \boldsymbol{I}_p$. Integrating out $(b_g, \boldsymbol{w}_g)$ yields a marginal likelihood (up to an additive constant):

$$\log p(\boldsymbol{y}_g \mid \boldsymbol{X}_g, \sigma_y^2, \lambda) = -\frac{1}{2} \left[(n_g - 1) \log \sigma_y^2 + \log |\boldsymbol{A}| - p \log \lambda + \frac{1}{\sigma_y^2} \left(s_{yy} - \boldsymbol{s}_{xy}^\top \boldsymbol{A}^{-1} \boldsymbol{s}_{xy} \right) \right].$$

This is the quantity implemented in PyMC as a `pm.Potential`, with all terms computed from the per-group sufficient statistics.

3.4 Middle ground for readability: explicit intercept, collapsed slopes

The fully collapsed formulation integrates out both $(b_g, \boldsymbol{w}_g)$, which is fast but can be hard to follow because the intercept does not appear as an explicit node in the PyMC model.

As a more readable middle ground, we can keep the per-group intercepts b_g as explicit PyMC variables, while still integrating out the (high-dimensional) slopes $\boldsymbol{w}_g$ analytically. This preserves most of the scalability

benefits (we still avoid sampling a large $\boldsymbol{w}$ tensor), but makes it easier to extend the model with intercept pooling later (e.g., cluster-level intercepts).

In code this corresponds to `--intercept-mode explicit` in `scripts/pipelines/train_rt_pymc_collap`

For a group g with uncentered design $\boldsymbol{X}_g \in \mathbb{R}^{n_g \times p}$ and response $\boldsymbol{y}_g \in \mathbb{R}^{n_g}$, define $\boldsymbol{A} = \boldsymbol{X}_g^\top \boldsymbol{X}_g + \lambda \boldsymbol{I}_p$ and residual $\boldsymbol{r}_g = \boldsymbol{y}_g - b_g \mathbf{1}$. Integrating out $\boldsymbol{w}_g$ yields:

$$\log p(\boldsymbol{y}_g \mid \boldsymbol{X}_g, b_g, \sigma_y^2, \lambda) = -\frac{1}{2}\left[n_g \log \sigma_y^2 + \log|\boldsymbol{A}| - p \log \lambda + \frac{1}{\sigma_y^2}\left(\boldsymbol{r}_g^\top \boldsymbol{r}_g - (\boldsymbol{X}_g^\top \boldsymbol{r}_g)^\top \boldsymbol{A}^{-1}(\boldsymbol{X}_g^\top \boldsymbol{r}_g)\right)\right].$$

All required terms can still be computed from per-group sufficient statistics:

$$\boldsymbol{r}_g^\top \boldsymbol{r}_g = \boldsymbol{y}_g^\top \boldsymbol{y}_g - 2b_g(\mathbf{1}^\top \boldsymbol{y}_g) + n_g b_g^2, \quad \boldsymbol{X}_g^\top \boldsymbol{r}_g = \boldsymbol{X}_g^\top \boldsymbol{y}_g - b_g(\boldsymbol{X}_g^\top \mathbf{1}).$$

3.5 Per-group posterior coefficients and predictive intervals

Conditioned on (σ_y^2, λ), the posterior for slopes is:

$$\boldsymbol{w}_g \mid \boldsymbol{y}_g, \boldsymbol{X}_g, \sigma_y^2, \lambda \sim \mathcal{N}\left(\hat{\boldsymbol{w}}_g, \sigma_y^2 \boldsymbol{A}^{-1}\right), \qquad \hat{\boldsymbol{w}}_g = \boldsymbol{A}^{-1}\boldsymbol{s}_{xy}.$$

The intercept is a derived random variable:

$$b_g = \bar{y}_g - \bar{\boldsymbol{x}}_g^\top \boldsymbol{w}_g,$$

so:

$$\mathbb{E}[b_g] = \bar{y}_g - \bar{\boldsymbol{x}}_g^\top \hat{\boldsymbol{w}}_g, \quad \mathrm{Var}(b_g) = \sigma_y^2\left(\frac{1}{n_g} + \bar{\boldsymbol{x}}_g^\top \boldsymbol{A}^{-1}\bar{\boldsymbol{x}}_g\right), \quad \mathrm{Cov}(b_g, \boldsymbol{w}_g) = -\sigma_y^2 \boldsymbol{A}^{-1}\bar{\boldsymbol{x}}_g.$$

We write these as a full coefficient covariance $\widehat{\Sigma}_{\beta,g}$ for $\boldsymbol{\beta}_g = [b_g; \boldsymbol{w}_g]$, stored in `Stage1CoeffSummaries` as `beta_cov`.

For a new row with features $\boldsymbol{x}$ (after the stored global transform), define $\boldsymbol{x}_1 = [1; \boldsymbol{x}]$. The posterior predictive distribution is approximated as Normal:

$$y^* \mid \boldsymbol{x}, \text{data} \approx \mathcal{N}\left(\boldsymbol{x}_1^\top \hat{\boldsymbol{\beta}}_g,\ \sigma_y^2 + \boldsymbol{x}_1^\top \widehat{\Sigma}_{\beta,g}\boldsymbol{x}_1\right).$$

3.6 Planned extension: shared chemistry intercept hierarchy (partial collapse)

Problem. The current baseline is evaluated on *seen groups* only: we require that the group key $g = (\texttt{species_cluster}, \texttt{comp_id})$ exists in the trained artifact, and we skip any rows whose group is missing. This is appropriate for benchmarking against per-group ridge, but it does not answer a common production need: **what do we predict for a new compound (or new group) that is not present in training?**

Motivation. We want a model that:

- stays competitive with the current ridge-equivalent baseline on *seen* (`species_cluster`, `comp_id`) groups;
- supports a principled backoff for *new compounds*, ideally leveraging chemistry similarity so "similar compounds" can share strength;
- produces usable predictive intervals for downstream peak assignment; and
- is easier to read and extend than a fully-collapsed derivation (i.e., make the hierarchy explicit in the PyMC graph while still keeping training scalable).

Design principle. Do *not* pool indiscriminately across `species_cluster` (clusters correspond to substantially different matrices and/or experimental conditions), but *do* allow chemistry information to be shared globally with strong shrinkage so that a chemical baseline can generalize to new compounds.

Plan. We keep the computationally critical part of the baseline (collapsing the ridge slopes) but make intercept structure explicit:

- Keep per-group intercepts b_g as explicit PyMC variables (as in `--intercept-mode explicit`).
- Add a hierarchical prior on b_g that decomposes it into: (i) a cluster-specific offset, and (ii) a shared chemistry baseline.
- Continue to integrate out the per-group slopes $\boldsymbol{w}_g$ analytically via a collapsed Gaussian likelihood (so we avoid sampling a large $G \times p$ coefficient tensor).

3.6.1 Revised model specification

Indices and data. Let:

- $c \in \{1, \ldots, C\}$ index `species_cluster` (matrix/condition clusters);
- $k \in \{1, \ldots, K\}$ index chemicals (`chem_id` = `compound` in the production CSV);
- $g \in \{1, \ldots, G\}$ index groups defined by $g = (\texttt{species_cluster}, \texttt{comp_id})$;
- $k(g)$ map each group g to its chemical id (resolved from `comp_id` $\rightarrow$ `chem_id` mapping);
- $c(g)$ map each group g to its cluster id;
- $\boldsymbol{x}_{gi} \in \mathbb{R}^p$ be run covariates (IS*/RS*/ES_* after the stored global transform);
- y_{gi} be `rt` for row i in group g.

Optionally, each chemical k has a fixed feature/embedding vector $\boldsymbol{z}_k \in \mathbb{R}^d$ (e.g., a ChemBERTa embedding or other chemistry descriptor).

Chemistry baseline (deterministic regression). To keep inference tractable while preserving chemistry-driven generalization, we use a deterministic chemistry baseline:

$$t_0 \sim \mathcal{N}(0, s_0^2),$$

$$t_k := t_0 + \boldsymbol{z}_k^\top \boldsymbol{\theta},$$

and place a Normal prior on regression weights:

$$\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, s_\theta^2 \boldsymbol{I}_d).$$

We use τ_t only as an additional uncertainty term for genuinely new chemicals at evaluation time, rather than sampling a separate random effect t_k for every chemical.

Cluster offsets (no cross-cluster pooling beyond a shared scale). Each cluster has its own additive offset:

$$\mu_c \mid \tau_\mu^2 \sim \mathcal{N}(0, \tau_\mu^2).$$

Group intercepts (the "seen-group head"). Each seen group g has an intercept b_g that shrinks toward the cluster offset plus the chemistry baseline:

$$b_g \mid \mu_{c(g)}, t_{k(g)}, \tau_b^2 \sim \mathcal{N}\big(\mu_{c(g)} + t_{k(g)}, \tau_b^2\big)\,.$$

Intuitively: t_k captures shared chemistry structure; μ_c captures cluster-wide RT shifts; τ_b governs how much we allow group-specific deviations around those shared terms.

Run-covariate slopes (ridge; collapsed). As in the baseline, each group has a slope vector:

$$\boldsymbol{w}_g \mid \sigma_y^2, \lambda \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma_y^2}{\lambda} \boldsymbol{I}_p\right),$$

and observations follow:

$$y_{gi} = b_g + \boldsymbol{x}_{gi}^\top \boldsymbol{w}_g + \varepsilon_{gi}, \qquad \varepsilon_{gi} \sim \mathcal{N}(0, \sigma_y^2).$$

Hyperpriors. We will use weakly-informative priors on scale parameters in RT units:

$$\sigma_y \sim \text{HalfNormal}(s_\sigma), \quad \tau_\mu \sim \text{HalfNormal}(s_\mu), \quad \tau_b \sim \text{HalfNormal}(s_b),$$

and a broad Normal prior on the global baseline:

$$t_0 \sim \mathcal{N}(0, s_0^2),$$

and, if chemistry features are used, a Normal prior on regression weights (as above). The ridge penalty λ can remain fixed (ridge-equivalent) or be learned as in `--lambda-mode learn`.

3.6.2 How we keep training scalable (partial collapse via sufficient statistics)

The PyMC graph will include $(\sigma_y, \lambda, \{\mu_c\}, \{t_k\}, \{b_g\}, \boldsymbol{\theta}, \tau.)$ explicitly. We still integrate out $\boldsymbol{w}_g$ analytically, so the likelihood contribution per group remains a collapsed Gaussian term.

To evaluate the collapsed likelihood without row-level data, we use per-group sufficient statistics. The current trainer already computes centered statistics from per-group means $(\bar{\boldsymbol{x}}_g, \bar{y}_g)$:

$$\boldsymbol{S}_{xx} = \boldsymbol{X}_c^\top \boldsymbol{X}_c, \quad \boldsymbol{s}_{xy} = \boldsymbol{X}_c^\top \boldsymbol{y}_c, \quad s_{yy} = \boldsymbol{y}_c^\top \boldsymbol{y}_c, \quad n_g.$$

From these we can recover the uncentered statistics needed for an explicit intercept b_g:

$$\boldsymbol{X}_g^\top \boldsymbol{X}_g = \boldsymbol{S}_{xx} + n_g \bar{\boldsymbol{x}}_g \bar{\boldsymbol{x}}_g^\top, \quad \boldsymbol{X}_g^\top \boldsymbol{y}_g = \boldsymbol{s}_{xy} + \bar{\boldsymbol{x}}_g (n_g \bar{y}_g),$$

$$\boldsymbol{X}_g^\top \mathbf{1} = n_g \bar{\boldsymbol{x}}_g, \quad \mathbf{1}^\top \boldsymbol{y}_g = n_g \bar{y}_g, \quad \boldsymbol{y}_g^\top \boldsymbol{y}_g = s_{yy} + n_g \bar{y}_g^2.$$

Given b_g, define the residual vector $\boldsymbol{r}_g = \boldsymbol{y}_g - b_g \mathbf{1}$ and let

$$\boldsymbol{A}_g = \boldsymbol{X}_g^\top \boldsymbol{X}_g + \lambda \boldsymbol{I}_p.$$

Integrating out $\boldsymbol{w}_g$ gives the per-group collapsed log-likelihood (up to a constant):

$$\log p(\boldsymbol{y}_g \mid \boldsymbol{X}_g, b_g, \sigma_y^2, \lambda) = -\frac{1}{2}\left[n_g \log \sigma_y^2 + \log |\boldsymbol{A}_g| - p \log \lambda + \frac{1}{\sigma_y^2}\left(\boldsymbol{r}_g^\top \boldsymbol{r}_g - (\boldsymbol{X}_g^\top \boldsymbol{r}_g)^\top \boldsymbol{A}_g^{-1} (\boldsymbol{X}_g^\top \boldsymbol{r}_g) \right) \right].$$

This is the only "linear algebra heavy" component, and it remains unchanged from the baseline except that b_g is now explicit.

3.6.3 Inference and production artifacts

Inference strategy. This model introduces many explicit latent variables (b_g for each group, t_k for each chemical, and μ_c for each cluster). To keep training practical, we plan to use the same inference modes already supported by the trainer:

- MAP (fast default) for point estimates of hyperparameters and latent intercept terms;
- ADVI when we need approximate posterior draws to propagate uncertainty more fully; and
- NUTS only for debugging on small subsets (e.g., limited number of groups) to sanity-check geometry and priors.

In all cases, the slopes $\boldsymbol{w}_g$ remain collapsed analytically.

What must be saved for production use. For *seen groups*, we can continue to materialize a `Stage1CoeffSummaries` artifact (per-group $\hat{\boldsymbol{\beta}}_g$ and covariance) so the existing streaming evaluator can produce predictions and intervals.

To support *backoff predictions for new compounds*, the production artifact must additionally include posterior summaries for:

- cluster offsets $\{\mu_c\}$ (and their uncertainty),
- chemical baselines $\{t_k\}$ (and their uncertainty),
- the global baseline t_0,
- chemistry regression weights $\boldsymbol{\theta}$ (and their uncertainty) if $\boldsymbol{z}_k$ features are used,
- the scale terms $(\tau_t, \tau_\mu, \tau_b)$ that govern cold-start uncertainty, and
- the global noise and ridge hyperparameters (σ_y, λ).

This can be implemented either by extending the existing `Stage1CoeffSummaries` format or by writing a companion artifact containing these additional parameters. The evaluator will need a corresponding "backoff" path for rows whose group key is missing.

3.6.4 Predictions and intervals (seen groups and backoff for new compounds)

Seen group prediction. For a seen group g, the predictive mean for a row with covariates $\boldsymbol{x}$ is:

$$\mathbb{E}[y^* \mid \boldsymbol{x}, \text{posterior}] \approx \mathbb{E}\left[b_g + \boldsymbol{x}^\top \hat{\boldsymbol{w}}_g\right],$$

where for fixed $(b_g, \sigma_y^2, \lambda)$ the ridge posterior mean is:

$$\hat{\boldsymbol{w}}_g = \boldsymbol{A}_g^{-1} \boldsymbol{X}_g^\top (\boldsymbol{y}_g - b_g \mathbf{1}).$$

We will propagate uncertainty by drawing from the PyMC posterior over the explicit variables and, for each draw, using the analytic Gaussian form of $\boldsymbol{w}_g \mid b_g, \sigma_y^2, \lambda$.

Predictive intervals. Conditional on $(b_g, \sigma_y^2, \lambda)$, the slopes have covariance $\sigma_y^2 \boldsymbol{A}_g^{-1}$ and the predictive variance for a new row is:

$$\mathrm{Var}(y^* \mid \boldsymbol{x}, b_g, \sigma_y^2, \lambda) \approx \sigma_y^2 + \boldsymbol{x}^\top (\sigma_y^2 \boldsymbol{A}_g^{-1}) \boldsymbol{x}.$$

Uncertainty in $(b_g, \mu_c, t_k, \boldsymbol{\theta}, \tau., \lambda, \sigma_y)$ is then incorporated by Monte Carlo over posterior draws, yielding calibrated intervals for downstream usage.

Backoff for new compounds (new `comp_id` and/or new chemical id). For a new compound not present in training, we do not have a learned b_g for its $(c, \texttt{comp_id})$ key. The hierarchy provides a principled backoff:

$$b_{\text{new}} \sim \mathcal{N}\left(\mu_c + t_k, \tau_b^2\right),$$

where t_k is either:

- drawn from the learned chemical baseline if the chemical id k was seen in training, or
- predicted from chemistry features via $t_k = t_0 + \boldsymbol{z}_k^\top \boldsymbol{\theta}$ for a genuinely new chemical, with additional uncertainty τ_t^2.

This supports cold-start predictions that share strength across chemically similar compounds while respecting matrix differences via μ_c. In the base formulation above, unseen-group slopes $\boldsymbol{w}$ revert to the ridge prior mean $\mathbf{0}$; if run-covariate response is required for cold-start accuracy, we can extend the model later with structured slope means (e.g., cluster-level or chemistry-gated slope means) while keeping the same partial-collapsed strategy.

4 Benchmark results (cap100 train → realtest eval)

All numbers below train on the full cap100 CSV and evaluate on the corresponding realtest CSV used by `scripts/run_rt_prod_eval.sh`. Evaluation is restricted to rows whose group key (`species_cluster`, `comp_i` exists in the coefficient artifact (seen groups only).

Lib	Model	n used	RMSE	$\text{cov}_{0.95}$
208	Lasso (eslasso)	915,327	0.0151	0.954
208	PyMC chem_hier (ADVI; explicit intercepts; λ fixed)	966,482	0.0094	0.984
209	Lasso (eslasso)	13,301,888	0.0094	0.993
209	PyMC chem_hier (ADVI; explicit intercepts; λ fixed)	13,596,681	0.0085	0.990

Table 1: Global metrics (trained on full cap100; evaluated on realtest; seen groups only).

Runtime (end-to-end). Training the chem_hier variant (explicit intercepts; ADVI with 5000 steps) took approximately 27m for lib208 and 48m for lib209 on the current workstation. Realtest evaluation took approximately 17s/214s using posterior predictive intervals (200 draws).

Predictive intervals ($\text{cov}_{0.95}$). The reported $\text{cov}_{0.95}$ is the fraction of test rows whose true RT falls inside the nominal 95% predictive interval. For PyMC chem_hier, we compute this via posterior predictive Monte Carlo (see `scripts/pipelines/eval_rt_pymc_posterior_predictive.py`). For the lasso baseline, the "interval" is a model-provided window; $\text{cov}_{0.95}$ is the fraction within that window.

4.1 Partially-collapsed chem_hier vs lasso baseline (realtest)

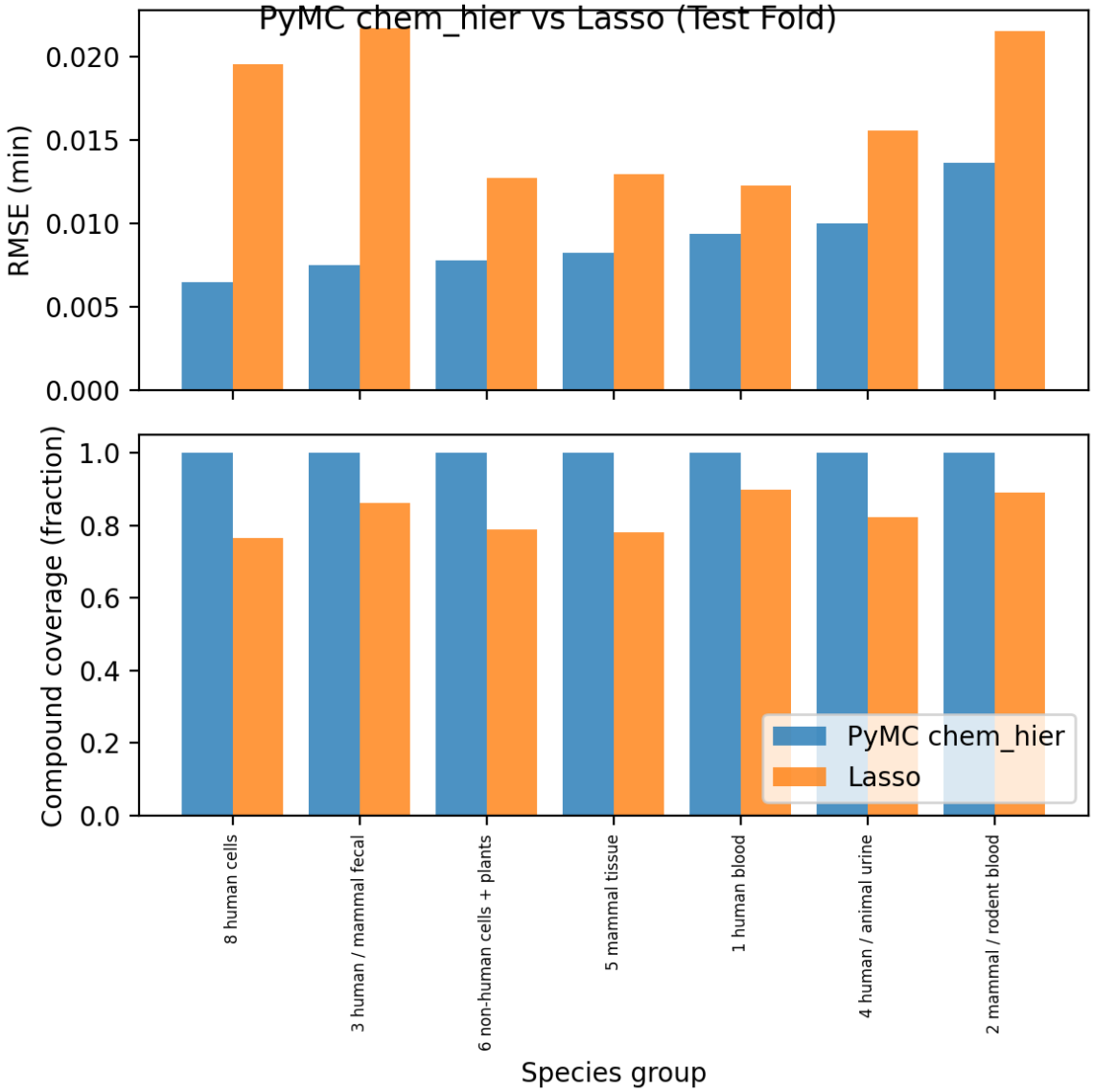
This comparison mirrors `scripts/run_rt_prod_eval.sh`: we aggregate RMSE by `species_group_raw` and show compound coverage (unique `compound` ids) alongside RMSE.

Species group	RMSE (chem_hier)	RMSE (lasso)	Ratio	Lasso cov.
1 human blood	0.0094	0.0123	0.77	0.90
2 mammal / rodent blood	0.0136	0.0215	0.63	0.89
3 human / mammal fecal	0.0075	0.0217	0.35	0.86
4 human / animal urine	0.0100	0.0156	0.64	0.82
5 mammal tissue	0.0083	0.0129	0.64	0.78
6 non-human cells + plants	0.0078	0.0127	0.61	0.79
8 human cells	0.0065	0.0196	0.33	0.76

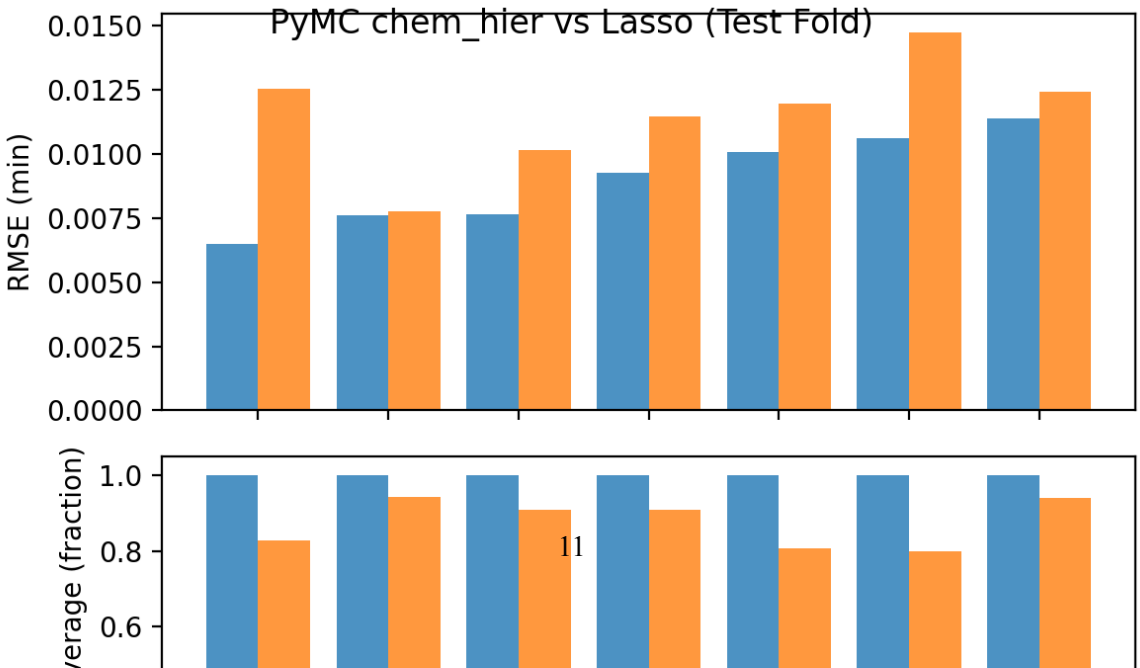
Table 2: lib208 realtest: RMSE by `species_group_raw` (PyMC chem_hier vs lasso).

Species group	RMSE (chem_hier)	RMSE (lasso)	Ratio	Lasso cov.
1 human blood	0.0076	0.0078	0.98	0.94
2 mammal / rodent blood	0.0077	0.0101	0.76	0.91
3 human / mammal fecal	0.0114	0.0124	0.92	0.94
4 human / fecal urine	0.0093	0.0115	0.81	0.91
5 mammal tissue	0.0101	0.0120	0.84	0.81
6 non-human cells + plants	0.0106	0.0147	0.72	0.80
8 human cells	0.0065	0.0126	0.52	0.83

Table 3: lib209 realtest: RMSE by `species_group_raw` (PyMC chem_hier vs lasso).



(a) lib208 (realtest).



4.2 Held-out chemicals (cap100 train; cold-start backoff on unseen groups)

Goal. Quantify what the chem_hier intercept hierarchy buys us for *unseen chemicals* (cold start), since realtest contains only seen (`species_cluster`, `comp_id`) groups for the cap100 training artifacts.

Experiment. For each library independently, we:

- randomly hold out 10% of unique `compound` (chem_id) values from the cap100 training CSV (seed 42);
- train chem_hier on the remaining rows (ADVI 5000 steps; λ fixed);
- evaluate on the held-out slice without requiring seen groups, so every row is scored via the backoff model (unseen (`species_cluster`, `comp_id`) by construction).

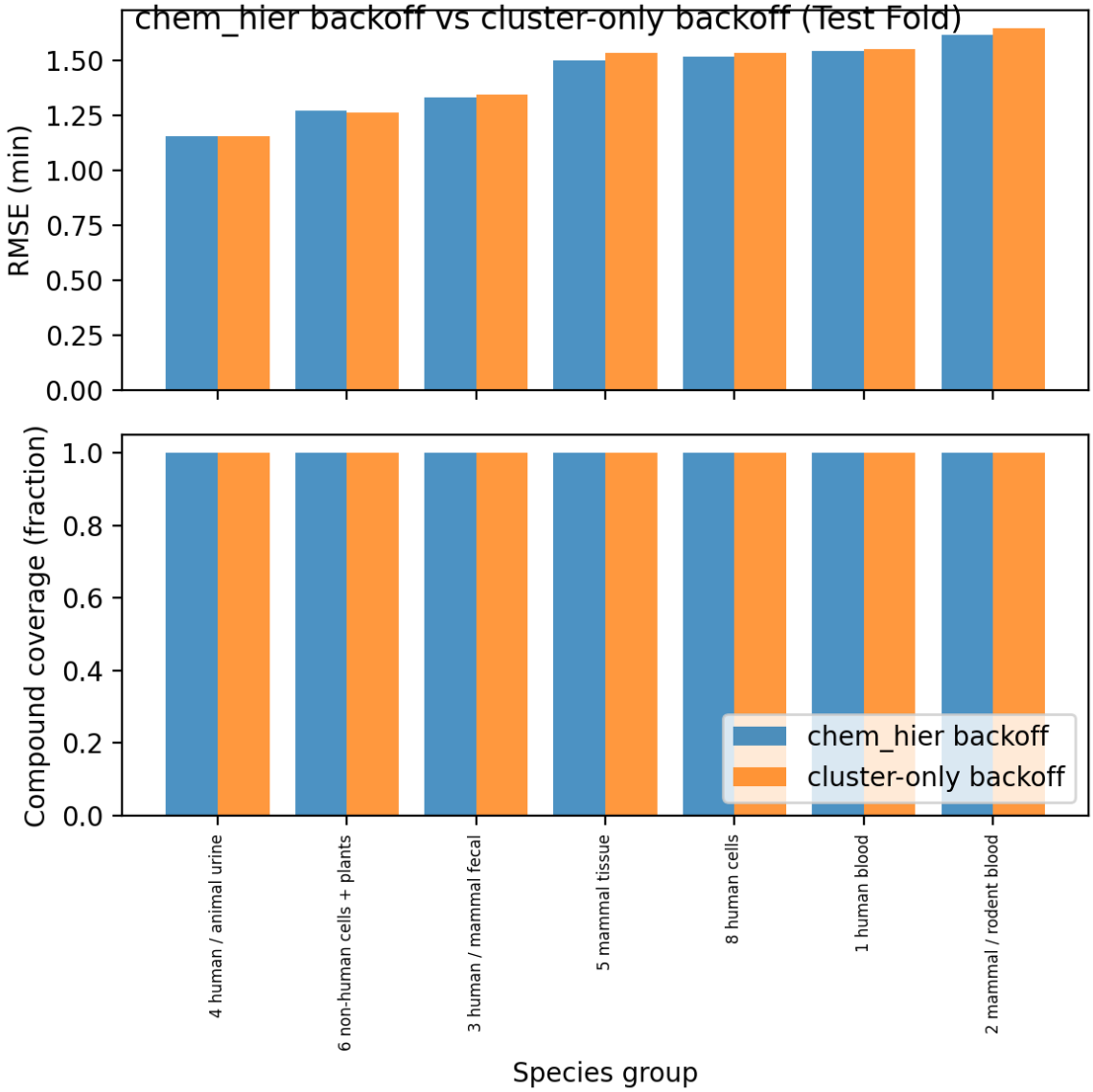
Backoff variants compared.

- **chem_hier backoff (chem)**: $b = \mu_c + t_0 + \boldsymbol{z}_k^\top \boldsymbol{\theta}$ for unseen chemicals.
- **cluster-only backoff**: $b = \mu_c + t_0$ (ignore chemistry; same uncertainty terms).

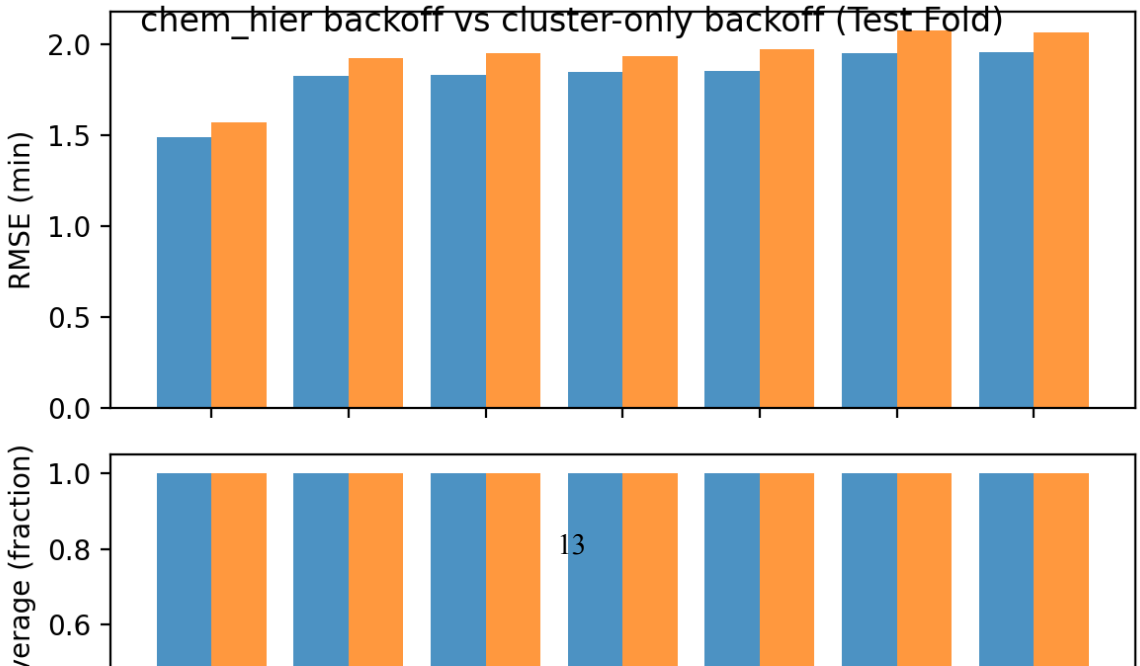
In both cases, unseen-group slopes revert to the ridge prior mean $\mathbf{0}$ (no shared slope head); this is a strict zero-shot setting.

Lib	Backoff mean	n used	RMSE	$\text{cov}_{0.95}$
208	chem_hier (chem)	121,092	1.3959	1.000
208	cluster-only	121,092	1.4078	0.999
209	chem_hier (chem)	357,488	1.8529	1.000
209	cluster-only	357,488	1.9604	0.999

Table 4: Held-out chemicals (10% chem_id holdout from cap100; backoff-only evaluation on the held-out slice).



(a) lib208 (held-out chemicals).



RMSE by species_group_raw (held-out chemicals).

5 Reproduction commands (cap100 train → realtest eval)

5.1 Per-species-group comparison (chem_hier vs lasso; realtest)

```
# Train chem_hier (cap100)
poetry run python scripts/pipelines/train_rt_pymc_collapsed_ridge.py \
  --data-csv repo_export/lib208/cap100/merged_training_all_lib208_cap100_chemclass_
  --include-es-all --lambda-mode fixed --lambda-slopes 1e-3 \
  --intercept-mode explicit --intercept-prior chem_hier \
  --chem-embeddings-path resources/metabolites/embeddings_chemberta_pca20.parquet \
  --chem-feature-center global \
  --method advi --advi-steps 5000 --advi-draws 200 \
  --output-dir output/tmp_pymc_chem_hier_lib208_cap100_advi --seed 42

poetry run python scripts/pipelines/train_rt_pymc_collapsed_ridge.py \
  --data-csv repo_export/lib209/cap100/merged_training_all_lib209_cap100_chemclass_
  --include-es-all --lambda-mode fixed --lambda-slopes 1e-3 \
  --intercept-mode explicit --intercept-prior chem_hier \
  --chem-embeddings-path resources/metabolites/embeddings_chemberta_pca20.parquet \
  --chem-feature-center global \
  --method advi --advi-steps 5000 --advi-draws 200 \
  --output-dir output/tmp_pymc_chem_hier_lib209_cap100_advi --seed 42

# Lasso baseline aggregated by species_group_raw
poetry run python scripts/pipelines/eval_rt_lasso_baseline.py \
  --train-output-dir output/tmp_lasso_lib208_realtest --lib-id 208 \
  --test-csv repo_export/lib208/realtest/merged_training_realtest_lib208_chemclass_
  --label realtest

poetry run python scripts/pipelines/eval_rt_lasso_baseline.py \
  --train-output-dir output/tmp_lasso_lib209_realtest --lib-id 209 \
  --test-csv repo_export/lib209/realtest/merged_training_realtest_lib209_chemclass_
  --label realtest

# chem_hier aggregated by species_group_raw
poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib208_cap100_advi \
  --test-csv repo_export/lib208/realtest/merged_training_realtest_lib208_chemclass_
  --require-seen-group --write-by-species-group \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label realtest_seen

poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib209_cap100_advi \
```

```
  --test-csv repo_export/lib209/realtest/merged_training_realtest_lib209_chemclass_
  --require-seen-group --write-by-species-group \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label realtest_seen

# Side-by-side plot by species_group_raw
poetry run python scripts/pipelines/compare_rt_models_by_group.py \
  --hier-csv output/tmp_pymc_chem_hier_lib208_cap100_advi/results/rt_eval_ppc_by_sp
  --lasso-csv output/tmp_lasso_lib208_realtest/results/rt_eval_lasso_by_species_gro
  --hier-label "PyMC chem_hier" --lasso-label "Lasso" --label realtest \
  --output docs/models/images/rt_pymc_collapsed_ridge_report/rt_compare_rmse_by_spe

poetry run python scripts/pipelines/compare_rt_models_by_group.py \
  --hier-csv output/tmp_pymc_chem_hier_lib209_cap100_advi/results/rt_eval_ppc_by_sp
  --lasso-csv output/tmp_lasso_lib209_realtest/results/rt_eval_lasso_by_species_gro
  --hier-label "PyMC chem_hier" --lasso-label "Lasso" --label realtest \
  --output docs/models/images/rt_pymc_collapsed_ridge_report/rt_compare_rmse_by_spe
```

5.2 Held-out chemicals (10% chem_id holdout; cap100)

```
# Build a cold-start split (hold out chem_ids) from the cap100 training CSV
poetry run python scripts/pipelines/split_rt_prod_csv_holdout_chems.py \
  --data-csv repo_export/lib208/cap100/merged_training_all_lib208_cap100_chemclass_
  --output-dir output/rt_pymc_chem_hier_bench/holdout_chems/lib208_cap100_seed42_f
  --holdout-frac 0.1 --seed 42 --overwrite

poetry run python scripts/pipelines/split_rt_prod_csv_holdout_chems.py \
  --data-csv repo_export/lib209/cap100/merged_training_all_lib209_cap100_chemclass_
  --output-dir output/rt_pymc_chem_hier_bench/holdout_chems/lib209_cap100_seed42_f
  --holdout-frac 0.1 --seed 42 --overwrite

# Train chem_hier on the reduced train split
poetry run python scripts/pipelines/train_rt_pymc_collapsed_ridge.py \
  --data-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib208_cap100_seed42_frac
  --include-es-all --lambda-mode fixed --lambda-slopes 1e-3 \
  --intercept-mode explicit --intercept-prior chem_hier \
  --chem-embeddings-path resources/metabolites/embeddings_chemberta_pca20.parquet \
  --chem-feature-center global \
  --method advi --advi-steps 5000 --advi-draws 200 \
  --output-dir output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi --seed 42

poetry run python scripts/pipelines/train_rt_pymc_collapsed_ridge.py \
  --data-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib209_cap100_seed42_frac
  --include-es-all --lambda-mode fixed --lambda-slopes 1e-3 \
  --intercept-mode explicit --intercept-prior chem_hier \
  --chem-embeddings-path resources/metabolites/embeddings_chemberta_pca20.parquet \
```

```
  --chem-feature-center global \
  --method advi --advi-steps 5000 --advi-draws 200 \
  --output-dir output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi --seed 42

# Evaluate on held-out chem_ids using backoff (unseen groups), and compare to a cl
poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi \
  --backoff-npz output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi/models/c
  --test-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib208_cap100_seed42_fra
  --backoff-mean-mode chem \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label holdout_chems --write-by-species-group

poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi \
  --backoff-npz output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi/models/c
  --test-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib208_cap100_seed42_fra
  --backoff-mean-mode cluster_only \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label holdout_chems_cluster_only --write-by-species-group

poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi \
  --backoff-npz output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi/models/c
  --test-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib209_cap100_seed42_fra
  --backoff-mean-mode chem \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label holdout_chems --write-by-species-group

poetry run python scripts/pipelines/eval_rt_pymc_posterior_predictive.py \
  --train-output-dir output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi \
  --backoff-npz output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi/models/c
  --test-csv output/rt_pymc_chem_hier_bench/holdout_chems/lib209_cap100_seed42_fra
  --backoff-mean-mode cluster_only \
  --max-test-rows 0 --chunk-size 50000 --n-ppc-draws 200 \
  --label holdout_chems_cluster_only --write-by-species-group

# Plot backoff comparison by species_group_raw
poetry run python scripts/pipelines/compare_rt_models_by_group.py \
  --hier-csv output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi/results/rt_
  --lasso-csv output/tmp_pymc_chem_hier_lib208_cap100_holdoutchems_advi/results/rt_
  --hier-label "chem_hier backoff" --lasso-label "cluster-only backoff" --label ho
  --output docs/models/images/rt_pymc_collapsed_ridge_report/rt_compare_rmse_by_sp

poetry run python scripts/pipelines/compare_rt_models_by_group.py \
  --hier-csv output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi/results/rt_
  --lasso-csv output/tmp_pymc_chem_hier_lib209_cap100_holdoutchems_advi/results/rt_
  --hier-label "chem_hier backoff" --lasso-label "cluster-only backoff" --label ho
```

```
--output docs/models/images/rt_pymc_collapsed_ridge_report/rt_compare_rmse_by_spe
```

6 Next steps

The partially-collapsed chem_hier model matches the collapsed ridge accuracy on *seen groups* while producing better-calibrated intervals and providing a backoff path for *unseen compounds*. The next iteration should focus on (1) making cold-start predictions useful and (2) bringing training time down:

- **Improve cold-start (unseen groups) beyond intercept-only backoff.** The held-out-chem experiment is strict zero-shot and currently reverts unseen-group slopes to $\mathbf{0}$, which yields poor RMSE. Add a shared slope mean (e.g., global and/or cluster-level) so unseen groups can inherit run-covariate response.
- **Re-run held-out evaluations after slope sharing.** Repeat the held-out-chem split and report RMSE/coverage (global and by `species_group_raw`) to quantify the lift from chemistry and the shared slope head.
- **Tune ADVI for speed/quality.** Sweep `--advi-steps` (and optionally `--advi-draws`) on cap5 first, then cap100, and record the time vs RMSE/coverage trade-off.
- **Optionally learn λ once the above is stable.** Enable `--lambda-mode learn` and check whether it improves intervals without harming seen-group RMSE.
- **Keep the benchmark comparison stable.** Continue reporting: (a) global metrics on realtest (seen groups only), (b) per-`species_group_raw` RMSE vs lasso (including lasso compound coverage), and (c) the held-out-chem experiment for cold-start.